
Leveraging Graph Geometry for Cold-Start Node Prediction

Anonymous Author(s)

Affiliation

Address

email

Abstract

Cold-start nodes pose a fundamental challenge for node prediction in graphs: at inference time, newly arriving nodes have no observed edges. This setting breaks a core assumption of most graph neural networks, which rely on observed adjacency at inference time. Recent cold-start methods that explicitly target this strict scenario often operate in a two-step manner: (i) retrieve a top- k neighborhood for the cold-start node, and (ii) aggregate information from the retrieved neighbors to predict the node label. A key design decision in this pipeline is the representation space in which the top- k retrieval is performed. This dramatically affects the quality of the recovered neighborhood, and consequently, the final prediction. We propose *SPARC*, a general framework that leverages the eigenfunctions of the graph Laplacian to estimate spectral embeddings, a representation that is theoretically known to capture the geometry of a graph. Since exact spectral embeddings are unavailable for cold-start nodes, *SPARC* learns a parametric map from node features to the Laplacian eigenspace, enabling us to approximate a cold-start node’s spectral embedding without access to its adjacency. We analyze why this spectral space provides superior neighborhood recovery compared to feature-space retrieval, and integrate *SPARC* into several recent state-of-the-art GNN architectures. Our code will be publicly available.

1 Introduction

Graphs are a fundamental representation for relational data, with applications in citation networks, social platforms, recommendation systems, and knowledge graphs. In these domains, node-level predictions typically rely on two complementary sources of information: the attributes of the target node and the structure of the graph around it. Modern graph learning methods [Hamilton et al., 2017, Veličković et al., 2018, Xu et al., 2018] exploit this structure by aggregating attribute information from neighboring nodes, allowing predictions to depend not only on the node itself but also on its relational context.

A key challenge arises when graph structure is unavailable at inference time. Existing graph learning settings differ in the amount of structural information available during training and test time. In the transductive setting [Kipf and Welling, 2016], all nodes and graph connectivity are observed during training, allowing models to exploit the full graph and often achieve strong performance. In the inductive setting [Hamilton et al., 2017], test nodes are unseen during training, but structural information becomes available at inference, so prediction can still leverage the graph. A more challenging regime arises when no structural information is available for the target nodes at all [Zheng et al., 2021, Hu et al., 2021]. In this paper, we focus on the stricter cold-start setting: test nodes are unseen during training and arrive at inference without any observed edges (Figure 1). This setting is particularly relevant in dynamic systems, where predictions must be made immediately when a new node appears, before any connectivity information has been observed.

Despite its practical importance, strict cold-start node prediction remains relatively underexplored. Existing methods [Zheng et al., 2021, Hu et al., 2021] that address this regime can often be viewed through a two-stage lens: first, construct a proxy neighborhood for the new node by retrieving top- k related training nodes; second, apply a downstream predictor over the retrieved neighborhood. Under this view, the central design choice is not only the classifier, but the representation space in which retrieval is performed. A natural baseline is to retrieve neighbors directly in the raw feature space, but feature similarity is often only a weak proxy for graph connectivity.

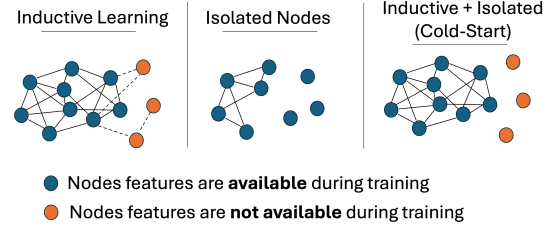


Figure 1: Problem setting for strict cold-start node prediction. Dashed edges denote connections known only at inference time. In the cold-start setting, the new node has features but no edges, requiring useful structural context to be recovered before prediction.

We address this bottleneck through spectral graph geometry. Spectral embeddings, derived from the eigenvectors of the graph Laplacian, provide a theoretically grounded representation of graph structure in which Euclidean distances reflect diffusion-based proximity on the graph [Coifman and Lafon, 2006]. This makes spectral space an attractive retrieval space for neighborhood construction: if a cold-start node could be placed in this space, one could retrieve structurally meaningful neighbors even without observing its edges, as we illustrate in this paper. However, exact spectral embeddings are not directly available in our setting: computing Laplacian eigenvectors can be infeasible for large graphs, and these eigenvectors are not defined for cold-start nodes whose adjacency is unknown.

To overcome this limitation, we propose SPARC, an inductive framework for strict cold-start graph learning. SPARC learns a parametric map from node features to approximate spectral coordinates of the observed training graph. During training, graph structure is used only through a spectral objective that encourages the learned representation to approximate the Laplacian eigenspace. At inference time, a cold-start node is embedded from its features alone, and its pseudo-neighborhood is retrieved in the learned spectral space. This enables graph-free inference while still injecting structural context into the downstream predictor.

A key advantage of SPARC is that it modifies neighborhood construction rather than the downstream predictor itself. Once a pseudo-neighborhood has been recovered in SPARC space, it can be supplied to different backbone models, including message-passing GNNs [Hamilton et al., 2017], GCNs [Zhu and Koniusz, 2021], and graph transformers [Chen et al., 2023].

Our contributions are as follows: (i) We propose SPARC, an inductive feature-to-spectral mapping that approximates graph spectral geometry for nodes without observed edges. (ii) We show that SPARC improves approximated-neighborhood recovery compared to feature-space and distillation-based alternatives. (iii) We demonstrate that SPARC improves strict cold-start node classification across datasets and can be integrated as a plug-in module across multiple graph backbones.

2 Related work

Graph learning. Most graph learning architectures require an observed adjacency structure, sampled neighborhood, or graph-derived context when making predictions. This includes message-passing GNNs such as GCN [Kipf and Welling, 2016], GraphSAGE [Hamilton et al., 2017], GAT [Veličković et al., 2018], and GIN [Xu et al., 2018], as well as more recent graph transformer and scalable graph-learning architectures such as GraphGPS [Rampášek et al., 2023], NodeFormer [Wu et al., 2023], NAGphormer [Chen et al., 2023], and Exphormer [Shirzad et al., 2023]. Although inductive models such as GraphSAGE can generalize to unseen nodes, they still require neighbors or graph-derived context for the target node at inference time. As a result, these methods do not directly apply to strict cold-start nodes that arrive with features but without observed incident edges.

Cold-start and graph-free node prediction. Strict cold-start node prediction studies the setting in which a test node must be classified from its features alone, without observed incident edges at inference time. Cold-Brew [Zheng et al., 2021] is the most closely related work to our setting:

it addresses nodes with incomplete or missing neighborhoods through teacher–student distillation, transferring structural information from a graph model to a feature-only student.

A broader line of work reduces inference-time dependence on message passing by training MLP-style graph predictors. Graph-MLP [Hu et al., 2021], GLNN [Zhang et al., 2021], SA-MLP [Chen et al., 2024], InfGraND [Eskandari et al., 2026], and ProGMLP [Lu et al., 2025] all aim to transfer graph-aware information into models that can perform graph-free or low-latency inference. Related work has also studied graph-free prediction for other tasks, such as Linkless Link Prediction [Guo et al., 2023], which distills relational knowledge for link prediction without graph-based inference.

Eigenfunction approximation and spectral geometry. Classical spectral methods require eigenvectors of a Laplacian defined on the observed graph, and thus cannot directly represent nodes that are absent at training time. A natural workaround is to learn eigenfunctions, or equivalently, a mapping into the Laplacian eigenspace, as a function of node features. SpectralNet [Shaham et al., 2018] follows this approach by optimizing a Rayleigh quotient objective to learn a parametric map that approximates leading Laplacian eigenvectors. Among existing eigenfunction/eigenvector approximation methods [Streicher et al., 2023, Ben-Ari et al., 2025], SpectralNet provides a scalable foundation for producing embeddings that generalize to unseen nodes.

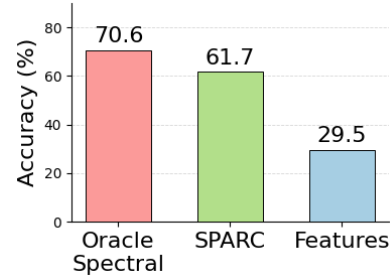
3 Notation and setup

We consider an attributed graph $\mathcal{G} = (\mathcal{V}, \mathcal{E}, X)$, where $\mathcal{V}$ is the node set, $\mathcal{E}$ is the edge set, $X \in \mathbb{R}^{n \times d}$ is the node-feature matrix. For node v_i , we write its feature vector as $x_i \in \mathbb{R}^d$. The adjacency matrix is $A \in \{0, 1\}^{n \times n}$, the degree matrix is D , and the normalized Laplacian is $L = I - D^{-1/2}AD^{-1/2}$.

We study strict inductive cold-start node classification. The node set is split into observed training nodes $\mathcal{V}_{\text{tr}}$ and cold-start nodes $\mathcal{V}_{\text{cold}}$, and the model only observes the training graph $\mathcal{G}_{\text{tr}} = (\mathcal{V}_{\text{tr}}, \mathcal{E}_{\text{tr}}, X_{\text{tr}})$. At test time, a node $v_{\text{cold}} \in \mathcal{V}_{\text{cold}}$ is given only through its features x_{cold} , with its incident edges hidden, and the goal is to predict its label. When datasets do not natively contain unseen nodes, we simulate this setting by holding out a subset of nodes and removing their incident edges from the observed graph.

4 Method: SPARC

We introduce SPARC, an inductive module for strict cold-start graph learning. In the strict cold-start setting, a test node arrives with features but without edges, making its graph neighborhood unavailable at inference time. SPARC addresses this by learning a feature-to-spectral map $F_\theta : \mathbb{R}^d \rightarrow \mathbb{R}^r$ that embeds nodes into a structure-aware representation. At inference time, a cold-start node is embedded using its features alone, and its missing neighborhood is approximated by retrieving nearby nodes in the learned representation. This approximated neighborhood can then be supplied to downstream graph models without changing their architecture.



4.1 Neighborhood approximation

Many graph learning models rely on neighborhood information for node-level prediction. However, in the strict cold-start setting, the target node has no edges at inference time, and therefore its true neighborhood is unavailable. We cast this problem as neighborhood approximation: given a representation function $\phi : \mathcal{V} \rightarrow \mathbb{R}^r$, we approximate the neighborhood of a cold-start node v_{cold} by retrieving its k nearest training nodes:

$$\hat{\mathcal{N}}_k^\phi(v_{\text{cold}}) = \text{kNN}(\phi(v_{\text{cold}}); \{\phi(u)\}_{u \in \mathcal{V}_{\text{tr}}}), \quad (1)$$

where nearest neighbors are computed using Euclidean distance.

Figure 2: Neighborhood recovery on Reddit dataset. In Table 1, we are extending this analysis across datasets.

A naive representation for a cold-start node is its raw feature vector, i.e., $\phi(v) = x_v$. This baseline is simple, scalable, and fully inductive, but feature similarity is only an indirect proxy for graph proximity. This limitation is also reflected in prior work on link prediction and feature heterophily, where connectivity can depend on structural context that is not fully captured by node-feature similarity alone [Zhang and Chen, 2018, Chamberlain et al., 2023, Zhu et al., 2024]. The Reddit graph provides a concrete example of this phenomenon. There, nodes are posts represented by bag-of-words features, while edges connect posts that receive comments from the same user. A single user may comment on multiple unrelated topics, so two posts can be adjacent in the graph even when their text features are quite different. In such cases, nearest neighbors in raw feature space do not correspond to structurally relevant neighbors.

To quantify this gap, Figure 2 reports neighborhood recovery on the Reddit dataset when retrieval is performed in feature space. For each node, we retrieve the top- k most similar training nodes using Euclidean distance in feature space and compare the retrieved set to the node’s true two-hop graph neighborhood. The feature-space bar in Figure 2 shows that raw features do not provide an accurate approximation of the graph neighborhood in this setting, yielding 29.5% accuracy.

This motivates using representations that are more directly tied to graph geometry. Spectral embeddings, obtained from the low-frequency eigenvectors of the graph Laplacian, provide such a representation: Euclidean distances in spectral coordinates reflect the geometry of the graph [Belkin and Niyogi, 2003, Coifman and Lafon, 2006]. We therefore also evaluate retrieval using exact spectral embeddings computed from the observed graph. We refer to this setting as *oracle spectral retrieval*, because these embeddings require access to the graph adjacency and are therefore not available for a strict cold-start node at inference time. The oracle spectral bar in Figure 2 shows that spectral coordinates provide a stronger approximation of the true graph neighborhood, with 70.6% accuracy.

The gap between feature-space retrieval and oracle spectral retrieval suggests the central goal of SPARC: learn an inductive map from node features to spectral-like coordinates. Such a map should preserve the neighborhood-relevant geometry captured by spectral embeddings while remaining usable for cold-start nodes.

4.2 Learning SPARC embeddings

SPARC learns an inductive feature-to-geometry map $F_\theta : \mathbb{R}^d \rightarrow \mathbb{R}^r$ that approximates the spectral geometry of the training graph. Given node features X_B in a mini-batch B , we denote the learned embeddings by $Z_B = F_\theta(X_B)$. Following SpectralNet [Shaham et al., 2018], we train F_θ using a spectral objective:

$$\mathcal{L}_{\text{spec}} = \frac{1}{m} \text{Tr} (Z_B^\top L_B Z_B), \quad \text{s.t.} \quad Z_B^\top Z_B = I,$$

where $m = |B|$ and L_B is the normalized batch Laplacian induced by the training graph. This objective encourages adjacent or strongly connected nodes to have smooth embeddings, while the orthogonality constraint prevents collapse. Following SpectralNet [Shaham et al., 2018], we enforce the orthogonality constraint by architectural design using an orthogonalization layer. By the Rayleigh–Ritz characterization, minimizing the Laplacian quadratic form under orthogonality recovers the low-frequency eigenspace of the graph Laplacian. Thus, the spectral loss encourages F_θ to learn a feature-based approximation of the oracle spectral coordinates.

This component is label-free: it depends only on the training graph structure and node features. Therefore, SPARC can be trained in an unsupervised manner and reused across multiple downstream tasks defined on the same graph. When labels are available, we optionally add a label-aware spectral regularizer inspired by constrained spectral clustering [Wang et al., 2012]. For each mini-batch, we define a pairwise constraint matrix $Q_B \in \mathbb{R}^{m \times m}$:

$$(Q_B)_{ij} = \begin{cases} +1, & y_i = y_j, \\ -1, & y_i \neq y_j, \\ 0, & \text{if no label constraint is used.} \end{cases}$$

The corresponding regularizer is $\mathcal{L}_{\text{con}} = -\frac{1}{m} \text{Tr} (Z_B^\top Q_B Z_B)$, so the full SPARC objective is

$$\mathcal{L}_{\text{SPARC}} = \mathcal{L}_{\text{spec}} + \lambda_Q \mathcal{L}_{\text{con}} = \frac{1}{m} \text{Tr} (Z_B^\top (L_B - \lambda_Q Q_B) Z_B),$$

where $\lambda_Q \geq 0$ controls the strength of the label-aware term. To understand the effect of regularizing the Laplacian L by subtracting Q , note that by definition, adjacent nodes have negative off-diagonal entries in L ; thus, subtracting Q makes nodes of the same class more “adjacent”, and nodes of different classes more “remote”. We ablate the contribution of this term in Appendix D.

The resulting representation can be interpreted as a shared geometry: the spectral term captures the manifold induced by graph connectivity, while the label-aware regularizer term aligns this geometry with label-level semantic structure. Crucially, both graph and label information are used only during training. At inference time, a cold-start node is embedded by a single forward pass, $z_{\text{cold}} = F_{\theta}(x_{\text{cold}})$, without requiring edges or labels. As illustrated in Figure 3, the graph is needed only during training through the loss computation; once the encoder is learned, inference depends only on node features.

Empirically, Figure 2 shows that SPARC substantially closes the gap between feature-space retrieval and oracle spectral retrieval. This indicates that the learned embeddings provide a strong approximation to neighborhood structure.

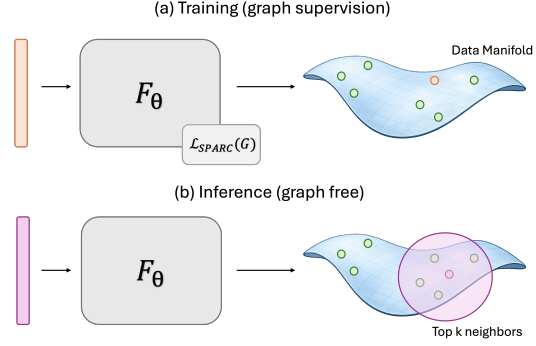


Figure 3: Overview of SPARC. The encoder learns a feature-to-spectral mapping during training. At inference time, pseudo-neighborhoods are retrieved for cold-start nodes without edges.

4.3 Aggregating approximated neighborhood for downstream tasks

After training, SPARC is used to construct a pseudo-neighborhood for each cold-start node. Given a cold-start node with features x_{cold} , we first compute $z_{\text{cold}} = F_{\theta}(x_{\text{cold}})$.

We then retrieve its k nearest training nodes in the learned SPARC representation:

$$\hat{\mathcal{N}}_k^{\text{SPARC}}(v_{\text{cold}}) = \text{kNN}(z_{\text{cold}}; \{z_u\}_{u \in \mathcal{V}_{\text{tr}}}).$$

The retrieved set serves as a pseudo-neighborhood that supplies structural context for the downstream predictor. SPARC modifies only the neighborhood construction step, not the downstream architecture. Therefore, it can be plugged into any method that consumes a neighborhood or context set. For message-passing GNNs such as GraphSAGE [Hamilton et al., 2017] or GCN [Zhu and Koniusz, 2021], the retrieved nodes are treated as pseudo-neighbors from which messages are aggregated. For graph transformers such as NAGphormer [Chen et al., 2023], the retrieved nodes define the context supplied to the transformer. In all cases, the backbone architecture and training objective are kept unchanged; SPARC only replaces the unavailable cold-start neighborhood with an inductively retrieved pseudo-neighborhood.

This separation makes SPARC a backbone-agnostic module for strict cold-start graph learning. It allows existing graph models to operate in settings where test nodes arrive without observed edges, while preserving the original model design.

5 Experiments

We evaluate SPARC under a strict cold-start node classification protocol, where test nodes are unseen during training and arrive at inference time without observed incident edges. Our experiments are designed to answer four questions. First, whether the learned SPARC space allows us to recover true graph neighborhoods more accurately than raw features or distillation-based embeddings. Second, whether improved neighborhood recovery translates into higher cold-start classification accuracy. Third, whether SPARC can serve as a plug-in retrieval module across different graph backbones. Finally, to assess the robustness of SPARC as the fraction of cold-start nodes increases.

5.1 Experimental setup

Strict cold-start protocol. Standard node classification benchmarks are defined on fixed graphs and therefore do not naturally contain cold-start nodes. To simulate the strict cold-start setting, we

Table 1: Neighborhood recovery accuracy across retrieval spaces. Oracle Spectral serves as a non-deployable graph-access reference.

Retrieval Space	Cora	Citeseer	Pubmed	Chameleon	Squirrel	Wiki-CS	Reddit	ogbn-products
Features	9.42%	17.05%	24.28%	39.94%	44.87%	75.37%	29.52%	50.48%
Cold-Brew	13.69%	19.30%	7.78%	49.82%	44.42%	58.95%	35.95%	47.66%
SPARC	62.69%	49.53%	31.26%	49.92%	45.02%	62.71%	61.66%	65.53%
Oracle Spectral	78.94%	80.09%	88.11%	97.63%	90.52%	81.25%	70.58%	94.80%

randomly select a subset of nodes, 10% by default, and remove them from the observed training graph. These nodes are treated as cold-start test nodes: their features remain available, but all incident edges are hidden at inference time. Thus, prediction cannot rely on observed test-time adjacency and must instead use either the node features alone or a recovered pseudo-neighborhood.

Datasets. We evaluate on citation networks, Wikipedia graphs, heterophilous web graphs, social graphs, and large-scale OGB benchmarks. Dataset statistics are summarized in Appendix Table 4. Cora, Citeseer, and Pubmed [Sen et al., 2008] are citation networks; Chameleon and Squirrel [Rozemberczki et al., 2019] are heterophilous Wikipedia web graphs; Wiki-CS [Mernyei and Cangea, 2020] is a Wikipedia-based computer science graph; Reddit [Hamilton et al., 2017] is a post interaction graph; and ogbn-products [Hu et al., 2020] is a large-scale OGB node classification benchmark.

Baselines and evaluation. Existing approaches for strict cold-start prediction can be broadly grouped into two categories: methods that rely on feature-only prediction, and methods that transfer graph information into feature-based predictors through distillation. Accordingly, we compare SPARC against two representative baselines. The feature-only baseline predicts from the target node features alone. Cold-Brew [Zheng et al., 2021] represents the distillation-based family: it transfers information from a graph model into an MLP-style predictor and can therefore be applied when test-time edges are unavailable. Several additional GNN-to-MLP methods are related to this setting, but are not included as baselines because they do not provide code. We therefore use Cold-Brew as the closest reproducible distillation baseline for the strict cold-start setting studied in this paper. Finally, to evaluate whether SPARC is backbone-agnostic, we integrate it with additional well-established graph backbones, including GraphSAGE [Hamilton et al., 2017], Spectral-GCN [Zhu and Koniusz, 2021], and NAGphormer [Chen et al., 2023], as reported in Section 5.2.

5.2 Neighborhood approximation quality

A central hypothesis of SPARC is that strict cold-start prediction depends on the quality of the pseudo-neighborhood recovered for each node. We therefore first evaluate the retrieval space directly, independently of the downstream classifier. For each node, we retrieve the top- k nearest training nodes in a given representation space and measure how well the retrieved set matches the node’s true neighborhood in the graph. Specifically, we measure retrieval accuracy as the fraction of retrieved nodes that belong to the ground-truth two-hop neighborhood of the cold-start node. We use the two-hop neighborhood because many downstream graph predictors rely on broader local context rather than only immediate neighbors. Further details on the metric definition are provided in Appendix E.2.

Table 1 compares retrieval in raw feature space, Cold-Brew, SPARC, and oracle spectral embeddings. The oracle spectral representation is computed using access to the full graph and is included only as a non-deployable reference for the quality of graph-spectral retrieval. Across datasets, retrieval in raw feature space and Cold-Brew embeddings is often limited, indicating that feature-based similarity alone does not always recover the local graph context needed for cold-start prediction. In contrast, SPARC achieves the strongest retrieval performance among deployable cold-start methods on most datasets. This supports the main mechanism of SPARC: learning an inductive approximation to spectral graph geometry provides a more informative space for pseudo-neighborhood construction than retrieving directly from raw features or from a distillation-based representation.

The oracle spectral results also provide useful context. They show that spectral geometry can recover substantially more of the true neighborhood when graph access is available, confirming that spectral space is a strong target for neighborhood approximation. At the same time, the gap between SPARC and Oracle Spectral reflects the difficulty of the strict cold-start setting: SPARC must approximate

Table 2: Strict cold-start node classification accuracy. All methods use the same downstream aggregator. FCR is a dataset-level diagnostic; higher FCR indicates stronger feature-only signal. $\Delta_{\text{SPARC-Features}}$ reports absolute improvement in percentage points.

Method	Cora	Citeseer	Pubmed	Chameleon	Squirrel	Wiki-CS	Reddit	ogbn-products
Features	69.74 $\pm$ 1.29	72.07 $\pm$ 0.99	86.41 $\pm$ 0.87	48.25 $\pm$ 0.42	31.92 $\pm$ 0.23	80.51 $\pm$ 1.39	75.03 $\pm$ 0.22	75.35 $\pm$ 0.46
Cold-Brew	72.69 $\pm$ 0.24	71.77 $\pm$ 0.87	85.80 $\pm$ 1.13	39.04 $\pm$ 0.83	31.73 $\pm$ 0.72	78.72 $\pm$ 1.19	73.70 $\pm$ 0.29	79.27 $\pm$ 0.39
SPARC	77.86 $\pm$ 0.92	75.38 $\pm$ 0.25	86.51 $\pm$ 0.89	52.63 $\pm$ 0.43	37.12 $\pm$ 0.76	81.22 $\pm$ 0.36	78.54 $\pm$ 0.89	81.34 $\pm$ 0.19
FCR	5.73%	24.00%	83.42%	55.87%	53.92%	32.88%	8.50%	16.27%
Δ SPARC - Features (pp)	+8.12	+3.31	+0.10	+4.38	+5.20	+0.71	+3.51	+5.99

this geometry from node features alone, without observing edges for the test node. Thus, the goal of SPARC is not to match oracle spectral embeddings exactly, but to recover enough of their neighborhood structure to improve downstream cold-start prediction. The relative gains vary across datasets, which is expected. When raw features are already well aligned with graph neighborhoods, feature-space retrieval can be competitive. When this alignment is weaker, SPARC provides larger gains by learning a geometry shaped by the training graph rather than by raw feature similarity alone.

5.3 Strict cold-start node classification

We next evaluate whether improved neighborhood retrieval translates into better downstream classification. Table 2 reports classification accuracy on strict cold-start test nodes using a NAGphormer-based downstream aggregator; implementation details are provided in Appendix E.3. Since all methods use the same downstream aggregator and differ only in how the pseudo-neighborhood is constructed, differences in performance can be attributed primarily to the retrieval strategy.

SPARC consistently improves over the feature-based retrieval baseline, with particularly large gains on Cora, Chameleon, Squirrel, Reddit, and ogbn-products. These improvements suggest that raw feature similarity alone does not always recover the structural context needed for accurate cold-start prediction. By retrieving neighbors in a learned spectral space, SPARC provides the downstream classifier with more informative contextual nodes.

SPARC also outperforms Cold-Brew on all datasets in our evaluation. We note that Cold-Brew is sometimes weaker than the feature-based retrieval baseline. This does not contradict the results reported in Cold-Brew [Zheng et al., 2021], where the main comparison is against an MLP that predicts from the node features alone. In contrast, our Features baseline is stronger: it retrieves a pseudo-neighborhood using feature-space similarity and then applies downstream aggregator used by SPARC. Therefore, this baseline uses both the cold-start node features and the features of similar training nodes. When feature similarity is already predictive of the task, this retrieval-based baseline can outperform a distillation-based representation. The fact that SPARC improves over this stronger baseline indicates that the learned spectral space captures useful structural information beyond raw feature similarity.

To interpret when SPARC helps most, we also report the feature contribution ratio (FCR) introduced by Cold-Brew [Zheng et al., 2021]. FCR estimates the extent to which node features alone contribute to classification performance. Datasets with high FCR are more feature-dominant, leaving less room for improvement over feature-based retrieval. Conversely, lower FCR indicates that structural context plays a larger role, in which case SPARC tends to provide larger gains. For example, Pubmed has high FCR and SPARC yields only a small improvement over Features, whereas Cora and Reddit have low FCR and show larger gains. Further details on the FCR computation are provided in Appendix C.

5.4 SPARC as a plug-in retrieval module

SPARC modifies how structural context is constructed, rather than introducing a new downstream classifier. To test whether its benefit is architecture-specific, we integrate SPARC into several graph backbones and compare each model to its version with SPARC added. This experiment evaluates whether learned spectral retrieval can improve different downstream architectures.

For the base models, cold-start nodes are evaluated with only self-loops, since their incident edges are unavailable at inference time. As a result, the base performance reflects how well each architecture can classify the same cold-start nodes when no neighborhood information is provided. The variation in base accuracy across backbones highlights that different architectures depend on graph context to

Table 3: SPARC as a plug-in retrieval module across graph backbones. Δ denotes the absolute gain from adding SPARC-based pseudo-neighborhood retrieval.

Dataset	GraphSAGE			Spectral-GCN			NAGphormer		
	Base	+SPARC	Δ	Base	+SPARC	Δ	Base	+SPARC	Δ
Cora	61.99	71.21	+9.22	54.98	76.38	+21.40	55.80	77.86	+22.06
Citeseer	64.26	71.17	+6.91	42.34	69.36	+27.02	52.10	75.38	+23.28
Pubmed	84.53	86.05	+1.52	70.53	87.12	+16.59	69.10	86.51	+17.41
Chameleon	49.56	57.89	+8.33	35.08	49.12	+14.04	41.67	52.63	+10.96
Squirrel	34.61	35.38	+0.77	21.53	41.92	+20.39	31.54	37.12	+5.58
WikiCS	73.28	79.57	+6.29	23.84	81.53	+57.69	75.56	81.22	+5.66
Reddit	57.45	72.99	+15.54	12.44	78.13	+65.69	49.51	78.54	+29.03
ogbn-products	51.58	79.25	+27.67	9.45	83.33	+73.88	66.09	81.34	+15.25

different degrees. Models that rely more heavily on neighborhood information degrade more severely when the target node arrives without edges. Table 3 shows that adding SPARC improves performance across backbones [Hamilton et al., 2017, Zhu and Koniusz, 2021, Chen et al., 2023]. These gains indicate that the learned spectral retrieval space is not specific to a single downstream predictor. Instead, SPARC acts as a model-agnostic retrieval layer that supplies structurally meaningful pseudo-neighborhoods to different architectures.

The magnitude of the improvement varies by backbone and dataset. Architectures whose base versions suffer most from the absence of neighbors benefit the most from SPARC. For example, Spectral-GCN has low base performance on several datasets, reflecting its dependence on graph-defined structure, which becomes unavailable or uninformative for isolated cold-start nodes. After adding SPARC-based retrieval, its performance improves substantially. This suggests that SPARC can recover part of the missing structural context needed by graph-dependent architectures. Overall, these results support the use of SPARC as a plug-in retrieval module for strict cold-start prediction.

5.5 Robustness to the cold-start ratio

Finally, we evaluate how each method behaves as the strict cold-start setting becomes more severe. We vary the fraction of nodes held out as strict cold-start nodes over 5%, 10%, 20%, 50%, and 70%, and report classification accuracy on these nodes. As the cold-start ratio increases, the task becomes harder for two reasons: more test nodes must be classified without observed incident edges, and fewer nodes and edges remain available for learning the retrieval space.

Figure 4 shows that performance generally decreases as the cold-start ratio grows, reflecting the increasing difficulty of learning from a progressively smaller observed graph. Across this range, SPARC remains robust and preserves strong accuracy even in high cold-start regimes. The gains are especially pronounced on datasets such as Cora, Chameleon, Squirrel, and Reddit, where structural context plays an important role and feature-space retrieval is less sufficient. These results indicate that learning a spectral retrieval space helps SPARC maintain informative pseudo-neighborhoods as the amount of observed graph structure decreases.

6 Theoretical discussion

SPARC relies on the ability to map node features to its spectral embedding. It is therefore of interest to analyze the mathematical conditions under which this is possible.

Theorem 1 (informal: Neural Recovery of Manifold Eigenfunctions). *Assume the following:*

- 1. Features are smooth on the manifold:** *The observed data features represent a valid, non-collapsing projection (a smooth embedding) of an underlying continuous manifold.*
- 2. Leading spectral components are unambiguous:** *The manifold’s geometry yields a stable, distinctly separated set of k leading eigenfunctions (i.e., there is a strict spectral gap).*
- 3. Adjacencies are consistent with manifold metric:** *The empirical graph constructed during training consistently reflects the true intrinsic distances of the data.*

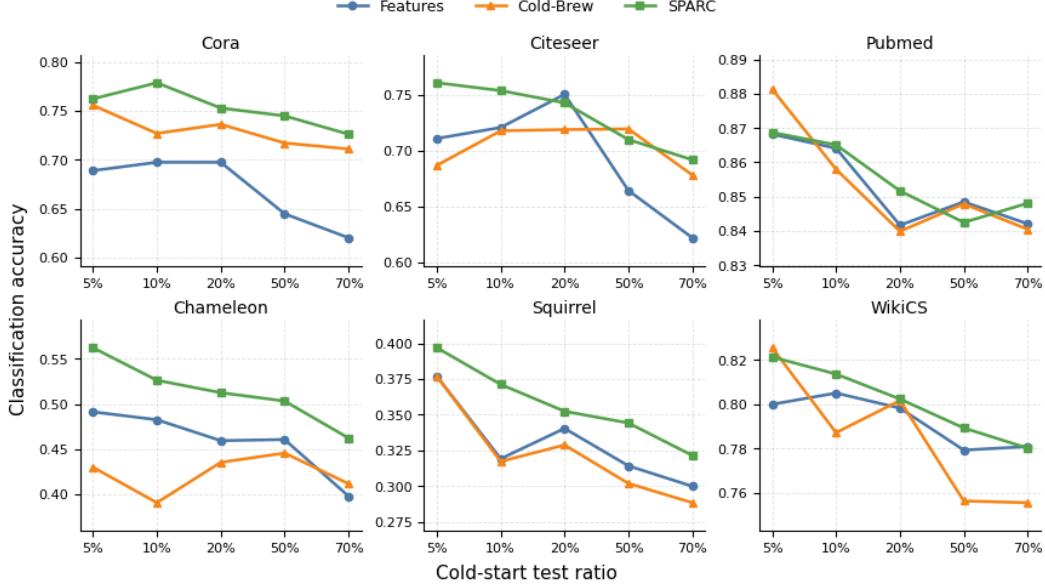


Figure 4: Sensitivity to the cold-start ratio. SPARC preserves strong performance as the observed training graph becomes smaller.

Then, there exists a standard feed-forward neural network that can map the observed data features directly to the true leading k eigenfunctions of the underlying manifold with arbitrary precision.

The formal statement and proof are provided in Appendix B. The intuition is as follows. Because the feature mapping does not "fold" or "tear" the underlying manifold, the intrinsic geometry is theoretically recoverable from the observed features X . By restricting our target to the leading k components where a spectral gap exists, and assuming the manifold geometry is reflected in the adjacencies, the target eigenspace is uniquely defined and highly robust to the sampling noise inherent in discrete graph approximations. Under these conditions, the target spectral map is a continuous function of the observed features. By the universal approximation theorem, a sufficiently expressive neural network can approximate this map arbitrarily well on compact domains.

7 Conclusion

Strict cold-start node prediction is challenging because newly arriving nodes have features but no observed edges, while most graph learning methods rely on neighborhood information at inference time. In this work, we framed this setting as a neighborhood-retrieval problem and identified the retrieval space as a central bottleneck.

We introduced SPARC, a geometry-aware retrieval framework that learns an inductive map from node features to approximate spectral embeddings of the observed graph. At inference time, SPARC embeds cold-start nodes from features alone and retrieves pseudo-neighborhoods in the learned spectral space, providing structural context without requiring test-time edges.

Across multiple datasets and graph backbones, SPARC improves neighborhood recovery and downstream cold-start classification compared to feature-space and distillation-based baselines. These results show that learned spectral geometry can serve as a plug-in mechanism for making existing graph models more effective in strict cold-start settings.

SPARC is most effective when node features contain enough information to recover aspects of the underlying graph geometry. Future work could extend this framework to settings with weaker feature-structure alignment, richer side information, and dynamic graphs where cold-start nodes gradually acquire edges.

References

- Mikhail Belkin and Partha Niyogi. Laplacian eigenmaps for dimensionality reduction and data representation. *Neural Computation*, 15(6):1373–1396, 2003. doi: 10.1162/089976603321780317.
- Nir Ben-Ari, Amitai Yacobi, and Uri Shoham. Generalizable spectral embedding with an application to umap, 2025. URL <https://arxiv.org/abs/2501.11305>.
- Benjamin Paul Chamberlain, Sergey Shirobokov, Emanuele Rossi, Fabrizio Frasca, Thomas Markovich, Nils Hammerla, Michael M. Bronstein, and Max Hansmire. Graph neural networks for link prediction with subgraph sketching, 2023. URL <https://arxiv.org/abs/2209.15486>.
- Jie Chen, Mingyuan Bai, Shouzheng Chen, Junbin Gao, Junping Zhang, and Jian Pu. SA-MLP: Distilling graph knowledge from GNNs into structure-aware MLP. *Transactions on Machine Learning Research*, 2024. ISSN 2835-8856. URL <https://openreview.net/forum?id=MZ2kKZc8m7>.
- Jinsong Chen, Kaiyuan Gao, Gaichao Li, and Kun He. Nagphormer: A tokenized graph transformer for node classification in large graphs, 2023. URL <https://arxiv.org/abs/2206.04910>.
- Ronald R. Coifman and Stéphane Lafon. Diffusion maps. *Applied and Computational Harmonic Analysis*, 21(1):5–30, 2006. ISSN 1063-5203. doi: <https://doi.org/10.1016/j.acha.2006.04.006>. URL <https://www.sciencedirect.com/science/article/pii/S1063520306000546>. Special Issue: Diffusion Maps and Wavelets.
- Amir Eskandari, Aman Anand, Elyas Rashno, and Farhana Zulkernine. Infgrand: An influence-guided gnn-to-mlp knowledge distillation, 2026. URL <https://arxiv.org/abs/2601.08033>.
- Zhichun Guo, William Shiao, Shichang Zhang, Yozen Liu, Nitesh V. Chawla, Neil Shah, and Tong Zhao. Linkless link prediction via relational distillation, 2023. URL <https://arxiv.org/abs/2210.05801>.
- William L. Hamilton, Rex Ying, and Jure Leskovec. Inductive representation learning on large graphs. *CoRR*, abs/1706.02216, 2017. URL <http://arxiv.org/abs/1706.02216>.
- Weihua Hu, Matthias Fey, Marinka Zitnik, Yuxiao Dong, Hongyu Ren, Bowen Liu, Michele Catasta, and Jure Leskovec. Open graph benchmark: Datasets for machine learning on graphs. *CoRR*, abs/2005.00687, 2020. URL <https://arxiv.org/abs/2005.00687>.
- Yang Hu, Haoxuan You, Zhecan Wang, Zhicheng Wang, Erjin Zhou, and Yue Gao. Graph-mlp: Node classification without message passing in graph. *CoRR*, abs/2106.04051, 2021. URL <https://arxiv.org/abs/2106.04051>.
- Thomas N. Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. *CoRR*, abs/1609.02907, 2016. URL <http://arxiv.org/abs/1609.02907>.
- Weigang Lu, Ziyu Guan, Wei Zhao, Yaming Yang, Yujie Sun, Zheng Liang, Yibing Zhan, and Dapeng Tao. Progmlp: A progressive framework for gnn-to-mlp knowledge distillation with efficient trade-offs, 2025. URL <https://arxiv.org/abs/2507.19031>.
- Péter Mernyei and Catalina Cangea. Wiki-cs: A wikipedia-based benchmark for graph neural networks. *CoRR*, abs/2007.02901, 2020. URL <https://arxiv.org/abs/2007.02901>.
- Ladislav Rampášek, Mikhail Galkin, Vijay Prakash Dwivedi, Anh Tuan Luu, Guy Wolf, and Dominique Beaini. Recipe for a general, powerful, scalable graph transformer, 2023. URL <https://arxiv.org/abs/2205.12454>.
- Benedek Rozemberczki, Carl Allen, and Rik Sarkar. Multi-scale attributed node embedding. *CoRR*, abs/1909.13021, 2019. URL <http://arxiv.org/abs/1909.13021>.
- Prithviraj Sen, Galileo Namata, Mustafa Bilgic, Lise Getoor, Brian Galligher, and Tina Eliassi-Rad. Collective classification in network data. *AI magazine*, 29(3):93–93, 2008.

425 Uri Shaham, Kelly Stanton, Henry Li, Ronen Basri, Boaz Nadler, and Yuval Kluger. Spectral-
426 net: Spectral clustering using deep neural networks. In *International Conference on Learning*
427 *Representations, ICLR*, 2018. URL https://openreview.net/forum?id=HJ_aoCyRZ.

428 Hamed Shirzad, Ameya Velingker, Balaji Venkatachalam, Danica J. Sutherland, and Ali Kemal
429 Sinop. Exphormer: Sparse transformers for graphs, 2023. URL <https://arxiv.org/abs/2303.06147>.

431 Or Streicher, Ido Cohen, and Guy Gilboa. Basis: Batch aligned spectral embedding space, 2023.
432 URL <https://arxiv.org/abs/2211.16960>.

433 Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua
434 Bengio. Graph attention networks, 2018. URL <https://arxiv.org/abs/1710.10903>.

435 Xiang Wang, Buyue Qian, and Ian Davidson. On constrained spectral clustering and its applications.
436 *CoRR*, abs/1201.5338, 2012. URL <http://arxiv.org/abs/1201.5338>.

437 Qitian Wu, Wentao Zhao, Zenan Li, David Wipf, and Junchi Yan. Nodeformer: A scalable graph
438 structure learning transformer for node classification, 2023. URL <https://arxiv.org/abs/2306.08385>.

440 Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. How powerful are graph neural
441 networks? *CoRR*, abs/1810.00826, 2018. URL <http://arxiv.org/abs/1810.00826>.

442 Muhan Zhang and Yixin Chen. Link prediction based on graph neural networks, 2018. URL
443 <https://arxiv.org/abs/1802.09691>.

444 Shichang Zhang, Yozen Liu, Yizhou Sun, and Neil Shah. Graph-less neural networks: Teaching old
445 mlps new tricks via distillation. *CoRR*, abs/2110.08727, 2021. URL <https://arxiv.org/abs/2110.08727>.

447 Wenqing Zheng, Edward W. Huang, Nikhil Rao, Sumeet Katariya, Zhangyang Wang, and Karthik
448 Subbian. Cold brew: Distilling graph node representations with incomplete or missing neighbor-
449 hoods. *CoRR*, abs/2111.04840, 2021. URL <https://arxiv.org/abs/2111.04840>.

450 Hao Zhu and Piotr Koniusz. Simple spectral graph convolution. In *International Conference on*
451 *Learning Representations*, 2021. URL <https://openreview.net/forum?id=CY05T-YjWZV>.

452 Jiong Zhu, Gaotang Li, Yao-An Yang, Jing Zhu, Xuehao Cui, and Danai Koutra. On the impact of
453 feature heterophily on link prediction with graph neural networks, 2024. URL <https://arxiv.org/abs/2409.17475>.

A Dataset statistics

Table 4 reports the size and feature dimensionality of the benchmark datasets used in the main experiments.

Table 4: Dataset statistics. The benchmark suite spans citation networks, Wikipedia graphs, social graphs, and large-scale OGB datasets.

Dataset	Nodes	Edges	Classes	Feats
Cora	2,708	5,429	7	1,433
Citeseer	3,312	4,732	6	3,703
Pubmed	19,717	44,338	3	500
Chameleon	2,277	36,101	5	2,325
Squirrel	5,201	217,073	5	2,089
Wiki-CS	11,701	216,123	10	300
Reddit	232,965	11.6M	41	602
ogbn-products	2.45M	61.9M	47	100

B Recovering the k -eigenspace from features

B.1 Formal definitions

To establish the relationship between the data manifold and the observed features, we must define the mapping that generates the features. This ensures the topological properties of the manifold are preserved in the feature space.

Definition 1 (Injective immersion). *A smooth map $g : \mathcal{M} \rightarrow \mathcal{X}$ between smooth manifolds is called an **injective immersion** if:*

1. *It is injective: $g(m_1) = g(m_2) \implies m_1 = m_2$.*
2. *Its differential $dg_m : T_m \mathcal{M} \rightarrow T_{g(m)} \mathcal{X}$ is injective for all $m \in \mathcal{M}$.*

Definition 2 (Smooth embedding). *A **smooth embedding** is an injective immersion $g : \mathcal{M} \rightarrow \mathcal{X}$ that is also a **homeomorphism** onto its image $g(\mathcal{M}) \subset \mathcal{X}$ (when the image is equipped with the subspace topology).*

Note: For a compact manifold $\mathcal{M}$, any injective immersion is automatically a smooth embedding.

B.2 Main result

By restricting our focus to the leading k eigenvectors, we introduce the requirement of a spectral gap. This gap ensures that the k -dimensional eigenspace we wish to approximate is uniquely defined and stable against perturbations in the feature space.

Theorem 2 (Existence of k -spectral feature mappings). *Let $(\mathcal{M}, \mathbf{g})$ be a smooth, compact n -dimensional Riemannian manifold. Let $\Delta_{\mathcal{M}}$ be the Laplace-Beltrami operator on $\mathcal{M}$ with eigenvalues $0 = \lambda_1 \leq \lambda_2 \leq \dots \leq \lambda_k < \lambda_{k+1}$.*

*Assume there exists a strictly positive **spectral gap** $\gamma = \lambda_{k+1} - \lambda_k > 0$. Let $\Phi : \mathcal{M} \rightarrow \mathbb{R}^k$ be a mapping whose components $\{\phi_i\}_{i=1}^k$ are the first k eigenfunctions of $\Delta_{\mathcal{M}}$.*

Let $\mathcal{X} = \mathbb{R}^d$ be a feature space and $g : \mathcal{M} \rightarrow \mathcal{X}$ be a smooth embedding. Provided the graph adjacencies used for training are consistent with the metric $\mathbf{g}$, then:

1. *There exists a continuous function $f : \mathcal{X} \rightarrow \mathbb{R}^k$ such that $f(g(m)) = \Phi(m)$ for all $m \in \mathcal{M}$.*
2. *For any $\epsilon > 0$, there exists a neural network f_{θ} such that:*

$$\sup_{m \in \mathcal{M}} \|f_{\theta}(g(m)) - \Phi(m)\| < \epsilon$$

Proof. 1. Regularity and Well-definedness: Since $(\mathcal{M}, \mathbf{g})$ is a smooth, compact Riemannian manifold, the Laplace-Beltrami operator $\Delta_{\mathcal{M}}$ is a self-adjoint elliptic operator. By elliptic regularity,

its eigenfunctions ϕ_i are smooth (C^∞) and continuous. Furthermore, the strict spectral gap $\gamma > 0$ guarantees that the k -dimensional subspace spanned by these eigenfunctions is isolated and uniquely defined (up to orthogonal transformations within eigenspaces of repeating eigenvalues). Consequently, the vector-valued function Φ is continuous and stable.

2. Continuity of the Inverse: By the definition of a smooth embedding, g is a homeomorphism onto its image $S = g(\mathcal{M}) \subset \mathbb{R}^d$. Therefore, the inverse map $g^{-1} : S \rightarrow \mathcal{M}$ is well-defined and continuous.

3. Composition and Extension: Define $h : S \rightarrow \mathbb{R}^k$ as $h = \Phi \circ g^{-1}$. As the composition of two continuous functions, h is continuous on the set S . Since $\mathcal{M}$ is compact, S is a compact (and thus closed) subset of $\mathbb{R}^d$. By the **Tietze Extension Theorem**, there exists a continuous function $f : \mathbb{R}^d \rightarrow \mathbb{R}^k$ such that the restriction $f|_S = h$.

4. Universal Approximation: Given that f is continuous and S is compact, the **Universal Approximation Theorem** for neural networks ensures that there exists a network f_θ with a k -dimensional output layer such that for any $\epsilon > 0$, $\|f_\theta(x) - f(x)\| < \epsilon$ for all $x \in S$. Substituting $x = g(m)$, we obtain $\sup_{m \in \mathcal{M}} \|f_\theta(g(m)) - \Phi(m)\| < \epsilon$. $\square$

Remark 1 (The importance of the spectral gap). *While the universal approximation theorem guarantees the existence of the network f_θ , calculating this network in practice relies on sampled data X . By the **Davis-Kahan Theorem**, the stability of an eigenspace under perturbation (such as the noise introduced by constructing a discrete graph Laplacian from finite data samples) is directly proportional to the spectral gap $\gamma = \lambda_{k+1} - \lambda_k$. If γ is large, the target mapping Φ remains robust, and the discrete eigenvectors used during the training of f_θ will closely align with the continuous eigenfunctions of the manifold. If $\gamma \rightarrow 0$, the subspace becomes highly sensitive to sampling noise, making it difficult for the network to consistently converge to the true intrinsic geometry of $\mathcal{M}$. By restricting the network’s output to the leading k components where a meaningful gap exists, SpectralNet inherently filters out high-frequency topological noise.*

C Feature contribution ratio details

Following the Cold Brew paper [Zheng et al., 2021], we report the *Feature Contribution Ratio* (FCR), which estimates the relative contribution of node features to the performance of a GNN by comparing the GNN against a feature-only MLP and label propagation (LP). Intuitively, higher FCR values indicate datasets where node features explain a larger portion of the GNN performance, while lower FCR values indicate datasets where graph structure plays a larger role.

Table 5: Feature Contribution Ratio (FCR) for each dataset, computed from the performance of the GNN, feature-only MLP, and label propagation (LP).

Dataset	GNN	MLP	LP	FCR
Cora	87.82	75.65	87.08	5.73%
Citeseer	74.47	68.77	72.67	24.00%
Pubmed	88.79	87.53	82.45	83.42%
Chameleon	64.91	45.17	39.91	55.87%
Squirrel	45.96	34.62	32.69	53.92%
WikiCS	82.99	78.46	80.77	32.88%
Reddit	95.22	72.83	93.72	8.50%
ogbn-products	90.12	72.35	88.88	16.27%

The results show substantial variation in FCR across datasets. For example, Pubmed obtains a high FCR of 83.42%, suggesting that node features already capture much of the signal needed for prediction. In contrast, Cora, Reddit, and ogbn-products obtain relatively low FCR values, indicating that graph structure contributes strongly beyond what is available from features alone.

D Ablation: effect of the task-aware loss

We ablate the contribution of the task-aware loss introduced in Section 4.2. The base variant of SPARC is trained only with the unsupervised spectral objective, $\mathcal{L}_{\text{spec}}$, which depends on the graph structure

and node features but does not use labels. This setting evaluates whether a purely structure-induced feature-to-geometry map is sufficient for strict cold-start node classification.

We also evaluate the complementary variant in which the encoder is trained only with the task-aware constraint term, $\mathcal{L}_{\text{con}}$, without the spectral objective. This variant uses labels to encourage nodes from the same class to be close and nodes from different classes to be separated, but it does not explicitly preserve the spectral geometry of the training graph. Finally, we compare both single-loss variants against the full SPARC objective, which combines the spectral objective with the task-aware constraint:

$$\mathcal{L}_{\text{SPARC}} = \mathcal{L}_{\text{spec}} + \lambda_Q \mathcal{L}_{\text{con}}.$$

Importantly, in all variants, graph structure and labels are used only during training. At inference time, all models follow the same strict cold-start protocol: the test node is embedded using its features alone, its pseudo-neighborhood is retrieved from the training nodes, and no edges or labels of the test node are observed.

Table 6 reports strict cold-start node classification accuracy for the three training objectives. The results show that the task-aware loss alone is not sufficient to obtain strong performance across datasets. Although it can improve over the purely spectral objective on some datasets, it may also underperform when label-level constraints are not grounded in the graph geometry. Conversely, the spectral objective alone provides a useful unsupervised geometry, but it is not always optimally aligned with the downstream classification task. The best results are obtained when both losses are combined, suggesting that SPARC benefits from learning a geometry that preserves the graph-induced manifold while also aligning it with task-level semantic structure.

Table 6: Ablation of the task-aware loss. We report strict cold-start node classification accuracy (%). ‘‘Spectral only’’ denotes SPARC trained only with $\mathcal{L}_{\text{spec}}$. ‘‘Task-aware only’’ denotes training only with $\mathcal{L}_{\text{con}}$, without the spectral objective. ‘‘Spectral + task-aware’’ denotes the full SPARC objective.

Dataset	Spectral Only	Task-Aware Only	Spectral + Task-Aware
Cora	69.74	72.80	77.86
Citeseer	69.32	70.47	75.38
Pubmed	83.22	85.40	86.51
Chameleon	46.59	41.23	52.63
Squirrel	34.71	33.08	37.12
WikiCS	77.39	79.06	81.22
Reddit	73.16	71.57	78.54
ogbn-products	75.23	76.34	81.34

E Implementation details and hyperparameters

This section provides the implementation details and hyperparameters used to train SPARC and to integrate it with the downstream graph backbones used in our experiments. Unless otherwise stated, the same hyperparameter configuration is used across datasets, and dataset-specific choices are selected using validation performance on the observed training graph.

E.1 SPARC encoder training

SPARC learns a feature-to-geometry map $F_\theta : \mathbb{R}^d \rightarrow \mathbb{R}^r$ that maps node features to a low-dimensional spectral representation. The encoder is trained only on the observed training graph under the strict cold-start split. Cold-start test nodes and their incident edges are not used during SPARC training.

The SPARC encoder is implemented as a multilayer perceptron (MLP). We train it using the objective described in Section 4, combining the unsupervised spectral loss with the optional task-aware constraint loss:

$$\mathcal{L}_{\text{SPARC}} = \mathcal{L}_{\text{spec}} + \lambda_Q \mathcal{L}_{\text{con}} = \frac{1}{m} \text{Tr} \left(Z_B^\top (L_B - \lambda_Q Q_B) Z_B \right),$$

where $Z_B = F_\theta(X_B)$ is the batch embedding matrix, L_B is the normalized batch Laplacian, Q_B is the label-constraint matrix, and λ_Q controls the strength of the task-aware loss. Unless otherwise

558 stated, $\lambda_Q = 0$ corresponds to the unsupervised SPARC variant used in the ablation, while $\lambda_Q > 0$
559 corresponds to the full task-aware SPARC model.

Table 7: Dataset-specific hyperparameters for SPARC encoder training. The hidden dimensions specify the MLP architecture ($d_{\text{in}} \rightarrow h_1 \rightarrow h_2 \rightarrow h_3 \rightarrow r$, with ReLU between hidden layers and tanh on the output), and r denotes the number of learned spectral dimensions, corresponding to the number of eigenvectors approximated by SPARC.

Parameter	Cora	Citeseer	Pubmed	Chameleon	Squirrel	Wiki-CS	Reddit	ogbn-products
Hidden dimensions	512, 512, 256	512, 512, 256	1024, 512, 256	1024, 1024, 512	1024, 1024, 512	1024, 1024, 512	256, 256, 128	1024, 1024, 512
Output dimension r	15	15	15	32	32	20	50	47
Initial learning rate	6.9e-4	1.1e-3	5e-4	1e-3	1e-3	5e-4	4.6e-4	1e-3
Minimum learning rate	1e-5	1e-5	1e-5	1e-7	1e-7	1e-5	1e-5	1e-5
LR-decay factor	0.5	0.5	0.5	0.1	0.1	0.5	0.5	0.5
Plateau patience (epochs)	20	20	20	10	10	20	20	20
Task-aware loss weight λ_Q	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5
Maximum training epochs	750	750	1000	1000	1000	1000	1000	1000

Table 8: Global SPARC training and retrieval settings used across datasets unless otherwise stated. Training stops early once ReduceLROnPlateau drives the learning rate to min_lr (Table 7); the maximum-epoch cap is therefore rarely reached on small graphs.

Hyperparameter	Value
Encoder architecture	4-layer fully-connected MLP
Hidden activation	ReLU
Output activation	tanh
Orthogonalization layer	QR-based, applied per forward pass
Optimizer	Adam
Laplacian for the spectral loss	symmetric normalized $L_{\text{sym}} = I - D^{-1/2} W D^{-1/2}$
Retrieval distance	Euclidean distance in SPARC space
Retrieval size k for neighborhood recovery	10 for small datasets, 20 for large-scale datasets
Cold-start ratio for main experiments	10%
Cold-start ratios for robustness analysis	5%, 10%, 20%, 50%, 70%

560 E.2 Pseudo-neighborhood construction

561 After SPARC training, we compute embeddings for all observed training nodes and for each cold-start
562 test node. For a cold-start node v_{cold} , we retrieve the k nearest observed training nodes in the learned
563 SPARC space:

$$\hat{\mathcal{N}}_k^{\text{SPARC}}(v_{\text{cold}}) = \text{kNN}(F_{\theta}(x_{\text{cold}}); \{F_{\theta}(x_u)\}_{u \in V_{\text{tr}}}),$$

564 using Euclidean distance. The retrieved nodes are treated as a pseudo-neighborhood for the cold-start
565 node. No true incident edges of cold-start nodes are used at inference time.

566 To evaluate neighborhood-recovery quality, we compare the retrieved pseudo-neighborhood to the
567 true two-hop neighborhood of the cold-start node in the original graph. Let $\mathcal{N}^{(2)}(v_{\text{cold}})$ denote the
568 true two-hop neighborhood of v_{cold} , restricted to candidate training nodes. For each cold-start node,
569 we compute

$$\text{NeighborhoodAcc}(v_{\text{cold}}) = \frac{|\hat{\mathcal{N}}_k(v_{\text{cold}}) \cap \mathcal{N}^{(2)}(v_{\text{cold}})|}{\min(k, |\mathcal{N}^{(2)}(v_{\text{cold}})|)}.$$

570 The final neighborhood-recovery score is obtained by averaging this quantity over all cold-start test
571 nodes. This metric measures the fraction of the retrievable true two-hop neighborhood recovered by
572 the predicted top- k set, while accounting for nodes whose true two-hop neighborhood contains fewer
573 than k candidates.

574 We use $k = 10$ for the smaller datasets and $k = 20$ for the large-scale datasets. The same value of k
575 is used consistently when comparing retrieval in feature space, Cold-Brew embedding space, SPARC
576 space, and oracle spectral space.

577 E.3 Downstream backbones

578 To evaluate whether SPARC is backbone-agnostic, we integrate the same retrieved pseudo-
 579 neighborhoods into GraphSAGE [Hamilton et al., 2017], Spectral-GCN [Zhu and Koniusz, 2021],
 580 and NAGphormer [Chen et al., 2023]. In all cases, the original backbone architecture and training
 581 objective are kept unchanged; SPARC only replaces the unavailable cold-start neighborhood with an
 582 inductively retrieved pseudo-neighborhood.

583 For GraphSAGE, the SPARC-retrieved nodes are treated as pseudo-neighbors for message passing.
 584 Thus, at inference time, a cold-start node aggregates information from its $k=10$ nearest training nodes
 585 in SPARC space, rather than from observed graph neighbors, which are unavailable under the strict
 586 cold-start protocol.

587 For Spectral-GCN, the standard spectral convolution is defined as

$$X^{l+1} = U g_\phi U^\top X^l,$$

588 where U contains the Laplacian eigenvectors and g_ϕ is a learnable spectral filter. This formulation
 589 cannot be directly applied to cold-start nodes, since they are not part of the original Laplacian and
 590 therefore do not have corresponding rows in U . With SPARC, we replace the fixed spectral basis U
 591 with the learned inductive approximation

$$\hat{U} = F_\theta(X) \in \mathbb{R}^{n \times r},$$

592 where r is the SPARC embedding dimension. The modified convolution becomes

$$X^{l+1} = \hat{U} g_\phi \hat{U}^\top X^l.$$

593 For a cold-start node with feature vector x_{cold} , we compute its SPARC embedding $u = F_\theta(x_{\text{cold}})$,
 594 retrieve its pseudo-neighborhood $\hat{\mathcal{N}}_k^{\text{SPARC}}(v_{\text{cold}})$, and apply the convolution locally:

$$x_{\text{cold}}^{l+1} = u g_\phi \hat{U}_{\hat{\mathcal{N}}}^\top X_{\hat{\mathcal{N}}}^l,$$

595 where $\hat{U}_{\hat{\mathcal{N}}} = F_\theta(X_{\hat{\mathcal{N}}})$. This allows the spectral convolution to be evaluated for cold-start nodes
 596 without recomputing the graph Laplacian or observing test-time edges.

597 For NAGphormer, each node is represented by a sequence of neighborhood-aggregated tokens,

$$\mathcal{T}(v_i) = [h_0(v_i), h_1(v_i), \dots, h_J(v_i)],$$

598 where $h_j(v_i)$ aggregates features from the j -hop neighborhood of v_i . Since cold-start nodes have
 599 no observed adjacency, their hop neighborhoods are unavailable. We therefore replace graph-based
 600 neighborhoods with SPARC-based retrieved neighborhoods. Specifically, for each cold-start node,
 601 we retrieve nearest training nodes in SPARC space and construct token representations by averaging
 602 increasingly large retrieved sets:

$$h_j(v_{\text{cold}}) = \frac{1}{|\hat{\mathcal{N}}_{m_j}^{\text{SPARC}}(v_{\text{cold}})|} \sum_{v \in \hat{\mathcal{N}}_{m_j}^{\text{SPARC}}(v_{\text{cold}})} x_v,$$

603 where m_j controls the number of retrieved nodes used for the j -th token. In our implementation,
 604 we use exponentially increasing context sizes, e.g., $m_j = 2^j$, to mimic the growth of multi-hop
 605 neighborhoods in the original NAGphormer construction. The resulting token sequence is then
 606 processed by the unchanged NAGphormer transformer aggregator.

607 E.4 Hyperparameter selection

608 Hyperparameters are selected using validation performance on the observed training graph under
 609 the same strict cold-start protocol. The test nodes are held out throughout model selection and are
 610 used only for final evaluation. For all methods, we use the same data splits, cold-start ratios, and
 611 evaluation protocol to ensure that differences in performance are attributable to the retrieval space and
 612 downstream aggregation strategy rather than to differences in data preprocessing or model selection.

613 **F Broader impact and limitations**

614 SPARC may improve prediction for newly arriving nodes in dynamic graph systems, such as citation,
615 recommendation, and ranking platforms. However, cold-start prediction in such systems may also
616 amplify biases present in node features or in the observed graph structure. In privacy-sensitive
617 domains, inferring pseudo-neighborhoods from features could also reveal or approximate hidden
618 relational information. Practical deployment should therefore include domain-specific fairness,
619 privacy, and robustness audits, especially when predictions affect users or access to opportunities.

620 **G OS and hardware**

621 The training procedures were executed on Rocky Linux 9.3, utilizing Nvidia GPUs including GeForce
622 GTX 1080 Ti and A100 80GB PCIe.

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [Yes]

Justification: The abstract and introduction state that the paper addresses strict cold-start node prediction by learning a feature-to-spectral mapping for pseudo-neighborhood retrieval, and the experiments directly evaluate neighborhood recovery, cold-start classification, robustness to cold-start ratio, and plug-in use across backbones.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [Yes]

Justification: The paper discusses the main limitation that SPARC relies on sufficient alignment between node features and the underlying graph geometry in order to learn a reliable feature-to-spectral mapping. This limitation is stated in the conclusion and is also reflected in the assumptions of the theoretical discussion.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [Yes]

Justification: The paper includes an informal theorem in the main text and provides the formal statement, assumptions, and proof in Appendix B. The assumptions include smooth feature embeddings, a spectral gap, and consistency between graph adjacencies and the manifold metric.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: The paper describes the strict cold-start protocol, datasets, baselines, evaluation metrics, SPARC training objective, pseudo-neighborhood construction, and downstream backbone integration. Appendix E further provides implementation details and hyperparameters needed to reproduce the main experimental setting.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [No]

Justification: Our code will be publicly made in the camera-ready version.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes]

Justification: The main experimental setup describes the strict cold-start split, datasets, baselines, and evaluation protocol, while Appendix E reports implementation details, SPARC hyperparameters, pseudo-neighborhood construction, and integration with GraphSAGE, Spectral-GCN, and NAGphormer.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.

672 • The experimental setting should be presented in the core of the paper to a level of detail
673 that is necessary to appreciate the results and make sense of them.
674 • The full details can be provided either with the code, in appendix, or as supplemental
675 material.

676 **7. Experiment statistical significance**

677 Question: Does the paper report error bars suitably and correctly defined or other appropriate
678 information about the statistical significance of the experiments?

679 Answer: [Yes]

680 Justification: We report mean and std accuracy for our main experiment.

681 **8. Experiments compute resources**

682 Question: For each experiment, does the paper provide sufficient information on the com-
683 puter resources (type of compute workers, memory, time of execution) needed to reproduce
684 the experiments?

685 Answer: [Yes]

686 Justification: see Appendix G

687 Guidelines:

688 • The answer [N/A] means that the paper does not include experiments.
689 • The paper should indicate the type of compute workers CPU or GPU, internal cluster,
690 or cloud provider, including relevant memory and storage.
691 • The paper should provide the amount of compute required for each of the individual
692 experimental runs as well as estimate the total compute.
693 • The paper should disclose whether the full research project required more compute
694 than the experiments reported in the paper (e.g., preliminary or failed experiments that
695 didn't make it into the paper).

696 **9. Code of ethics**

697 Question: Does the research conducted in the paper conform, in every respect, with the
698 NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

699 Answer: [Yes]

700 Justification: The work uses standard public graph benchmark datasets and proposes a
701 general graph-learning method. We are not aware of any aspect of the research that violates
702 the NeurIPS Code of Ethics.

703 Guidelines:

704 • The answer [N/A] means that the authors have not reviewed the NeurIPS Code of
705 Ethics.
706 • If the authors answer [No], they should explain the special circumstances that require a
707 deviation from the Code of Ethics.
708 • The authors should make sure to preserve anonymity (e.g., if there is a special consid-
709 eration due to laws or regulations in their jurisdiction).

710 **10. Broader impacts**

711 Question: Does the paper discuss both potential positive societal impacts and negative
712 societal impacts of the work performed?

713 [Yes]

714 Justification: The paper proposes a general method for strict cold-start node prediction in
715 graph learning. Potential positive impacts include improving prediction for newly arriving
716 nodes in dynamic systems such as recommendation platforms, citation networks, and other
717 relational datasets where immediate decisions are needed before graph connectivity is
718 observed, as discussed in Introduction. Additional impact in Appendix F

719 **11. Safeguards**

720 Question: Does the paper describe safeguards that have been put in place for responsible
721 release of data or models that have a high risk for misuse (e.g., pre-trained language models,
722 image generators, or scraped datasets)?

723 Answer: [N/A]
 724 Justification: The paper does not introduce a new dataset, benchmark, or separately released
 725 model asset as part of the submission.

726 **12. Licenses for existing assets**

727 Question: Are the creators or original owners of assets (e.g., code, data, models), used in
 728 the paper, properly credited and are the license and terms of use explicitly mentioned and
 729 properly respected?

730 Answer: [Yes]

731 Justification: To the best of our knowledge, all assets are credited.

732 **13. New assets**

733 Question: Are new assets introduced in the paper well documented and is the documentation
 734 provided alongside the assets?

735 Answer: [N/A]

736 Justification: No documentation provided alongside the assets.

737 Guidelines:

- 738 • The answer [N/A] means that the paper does not release new assets.
- 739 • Researchers should communicate the details of the dataset/code/model as part of their
 740 submissions via structured templates. This includes details about training, license,
 741 limitations, etc.
- 742 • The paper should discuss whether and how consent was obtained from people whose
 743 asset is used.
- 744 • At submission time, remember to anonymize your assets (if applicable). You can either
 745 create an anonymized URL or include an anonymized zip file.

746 **14. Crowdsourcing and research with human subjects**

747 Question: For crowdsourcing experiments and research with human subjects, does the paper
 748 include the full text of instructions given to participants and screenshots, if applicable, as
 749 well as details about compensation (if any)?

750 Answer: [N/A]

751 Justification: The paper does not involve crowdsourcing experiments or research with human
 752 subjects.

753 **15. Institutional review board (IRB) approvals or equivalent for research with human
 754 subjects**

755 Question: Does the paper describe potential risks incurred by study participants, whether
 756 such risks were disclosed to the subjects, and whether Institutional Review Board (IRB)
 757 approvals (or an equivalent approval/review based on the requirements of your country or
 758 institution) were obtained?

759 Answer: [N/A]

760 Justification: The paper does not involve human-subject experiments.

761 **16. Declaration of LLM usage**

762 Question: Does the paper describe the usage of LLMs if it is an important, original, or
 763 non-standard component of the core methods in this research? Note that if the LLM is used
 764 only for writing, editing, or formatting purposes and does *not* impact the core methodology,
 765 scientific rigor, or originality of the research, declaration is not required.

766 Answer: [N/A]

767 Justification: LLMs are not used as an important, original, or non-standard component of the
 768 proposed method, experiments, or scientific claims. Any use of LLMs for writing, editing,
 769 or formatting does not affect the core methodology or results.